

Reinforcement Learning

Mid Semester Project - 2026 B

This mid-semester project consolidates the theoretical and practical material of the course into one project on tabular reinforcement learning. You will solve two custom MiniGrid environments with Monte Carlo, SARSA, and Q-Learning, each implemented from scratch.

A note on this assignment. The sections below are topics to engage with, not a checklist. Many design choices (what to compare, how to present results, where to invest extra effort) are deliberately left to you.

1. Environments

- Built on the MiniGrid framework: a 2D grid-world with a triangular agent and a discrete action space.
- Both environments are fully defined in the template notebook (layout, dynamics, action space, helper methods, editable lines), which is the single source of truth. Do not import them from the standard MiniGrid library.

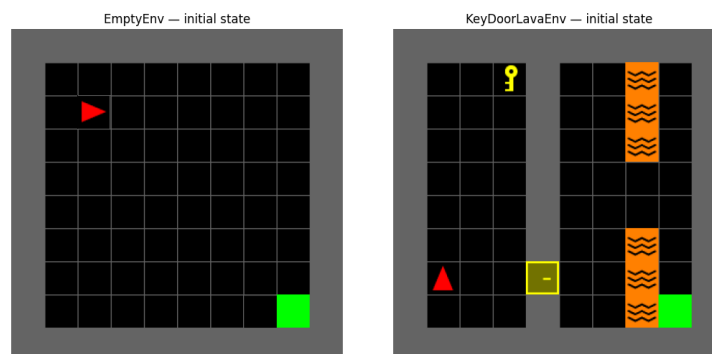
MiniGrid documentation: <https://minigrid.farama.org/environments/minigrid/>

1.1 EmptyEnv

- An empty room: the agent starts at a random cell and must reach a fixed goal under a sparse reward.
- Solve it exactly as defined in the notebook. Reward shaping is not allowed.
- You may set the episode length and scale the goal-reward magnitude at instantiation; nothing else may change.

1.2 KeyDoorLavaEnv

- A two-room grid with a locked door, a key, a column of lava, and a goal beyond the lava.
- The agent must collect the key, open the door, cross the lava through a safe gap, and reach the goal.
- The full specification (layout, dynamics, helper methods) is given in the notebook.



1.3 Reward shaping

- Reward shaping is optional for KeyDoorLavaEnv; its design is entirely up to you.
- You may use up to **two** shaping events: modify the reward for two specific events of your choosing, each a bonus or a penalty (e.g. “key picked up, reward = 50”).
- You may also add a small, uniform per-step penalty to encourage shorter solutions; it does not count toward the two events.
- **Forbidden:** distance- and geometry-based signals. No reward may depend on how close the agent is to the goal, key, door, or gap (including any tracked minimum distance), as this trivializes the task.
- Think about what your shaping actually encourages, and what it might encourage unintentionally.

1.4 Template Notebook

The template notebook shows how to load, run, and render both environments with random actions, and marks the editable lines. It is your starting point:

<https://colab.research.google.com/drive/1TapDlBEOo0MzBqCFvLfxm67BYTs9FDe?usp=sharing>

2. Analysis & Design

For each environment, engage with the topics below in both your notebook and report. You decide how deeply to treat each and which trade-offs to emphasize.

2.1 Understanding the environment

- Characterize each environment as an MDP: episodic or continuing, discrete or continuous state and action spaces, fully or partially observable.
- Point out the differences that matter for your approach, such as horizon length, reward sparsity, and the role of shaping.

2.2 State representation

- Explain your state representation: what information you keep and what you discard.
- State which of the environment actions your policy actually uses for each task, and which you leave out.
- Together these choices fix the Q-table size. State each environment’s state-space size and the resulting table size (number of states $\times$ number of actions).

2.3 Algorithms

- Solve each environment with all three algorithms: Monte Carlo, SARSA, and Q-Learning.
- Discuss the advantages and disadvantages of each in the context of the specific mission you are solving.
- Efficiency matters: prefer solutions that converge faster and require fewer steps at inference.

2.4 Hyperparameters & design choices

- Choose the learning rate, discount factor, exploration rate, and Q-table initialization deliberately, since they change results dramatically.
- Explain your reasoning and justify your exploration-exploitation strategy, emphasizing the choices that mattered most.

3. Training Results

For each environment and each algorithm, show how training progresses using graphs with the training episode on the x-axis. Plot:

- reward obtained per episode;
- number of steps taken per episode;
- success rate per episode (whether the agent reached the goal).

4. Inference Results

- After training, evaluate each learned policy greedily (no exploration) over several evaluation episodes.
- For each algorithm in each environment, report the average reward, the average number of steps to the goal, and the success rate.
- Present the results so the comparison across algorithms and environments is clear, for example as a table or a grouped bar chart.

5. Discussion

- Discuss the strengths and weaknesses of your approach for each algorithm and each environment.
- In a dedicated notebook cell, clearly state the best training and inference settings you found.

6. Note on Grading

Following these instructions thoroughly is the baseline for a strong grade. A perfect score requires going beyond the requirements; a well-motivated extension, a deeper analysis, or an insight that reveals genuine understanding of the material. The quality of that extra effort, not just its presence, is part of what we evaluate.

Try to "solve" each environment as fast as you can (fewer episodes/steps to convergence) - this is a small competition par.

7. Rules & Constraints

1. Implement the three algorithms yourself; existing reinforcement-learning libraries are not allowed.
2. Use only methods covered in the course; no deep reinforcement learning.
3. Edit the environment classes only where the notebook marks them editable, and place your own code below the “Your Code Below” divider. Do not otherwise change the environments, their action space, or layout.
4. Include the required training graphs even if you do not fully solve an environment.
5. Include short video clips of the agent partway through training and after it has converged (in the notebook).
6. Keep the notebook clean, with well-separated code and text cells. A messy notebook lowers your grade.
7. Do not modify the notebook after the deadline; a re-run or edited notebook may be disqualified.

8. Report

The report is the primary deliverable and must be fully self-contained. Write it as a professional scientific document from which a reader can understand your entire methodology and all of your results without opening the notebook or the code.

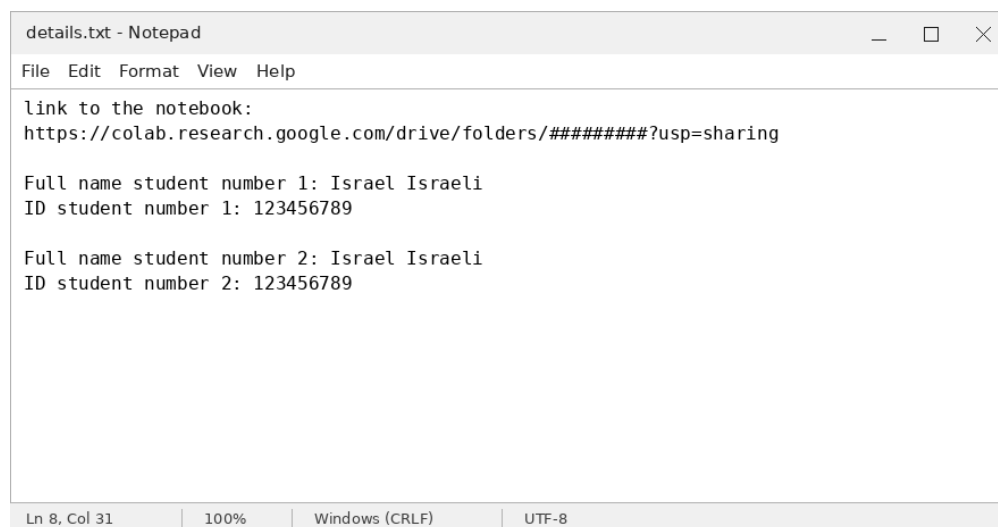
- Include every graph, table, numerical result, and discussion point the assignment asks for directly in the report itself.
- Name the file report_ID1_ID2.pdf, using both partners' ID numbers.
- Put the project title and both partners' full names and IDs at the top.
- Keep the report to 10 pages or fewer.

9. Submission

Submit the following files to the submission box:

1. details.txt: the link to your notebook and your partner details (see the template below).
2. report_ID1_ID2.pdf: your final report.
3. explainer.txt (optional): how to run your notebook, plus anything else a reader should know.

details.txt template:



```
details.txt - Notepad
File Edit Format View Help

link to the notebook:
https://colab.research.google.com/drive/folders/#####?usp=sharing

Full name student number 1: Israel Israeli
ID student number 1: 123456789

Full name student number 2: Israel Israeli
ID student number 2: 123456789

Ln 8, Col 31 | 100% | Windows (CRLF) | UTF-8
```

The notebook must be submitted as a Google Colab link the course team can run, and it **must already contain all of the outputs** from training and inference.

10. Team Size

The project is carried out in pairs. Submitting individually, without prior approval, results in a grade deduction.

11. Academic Integrity

- All submitted work must be your team's own. Copying code or answers from other students or teams, or sharing your own work, violates the university honor code, and you are responsible for protecting your work from copying.
- You may build on existing code (your own or external) if you cite it clearly and state what you changed and added. Unattributed use of others' work is a breach of academic integrity.
- Suspected violations are reported to the head of department and the university authorities.